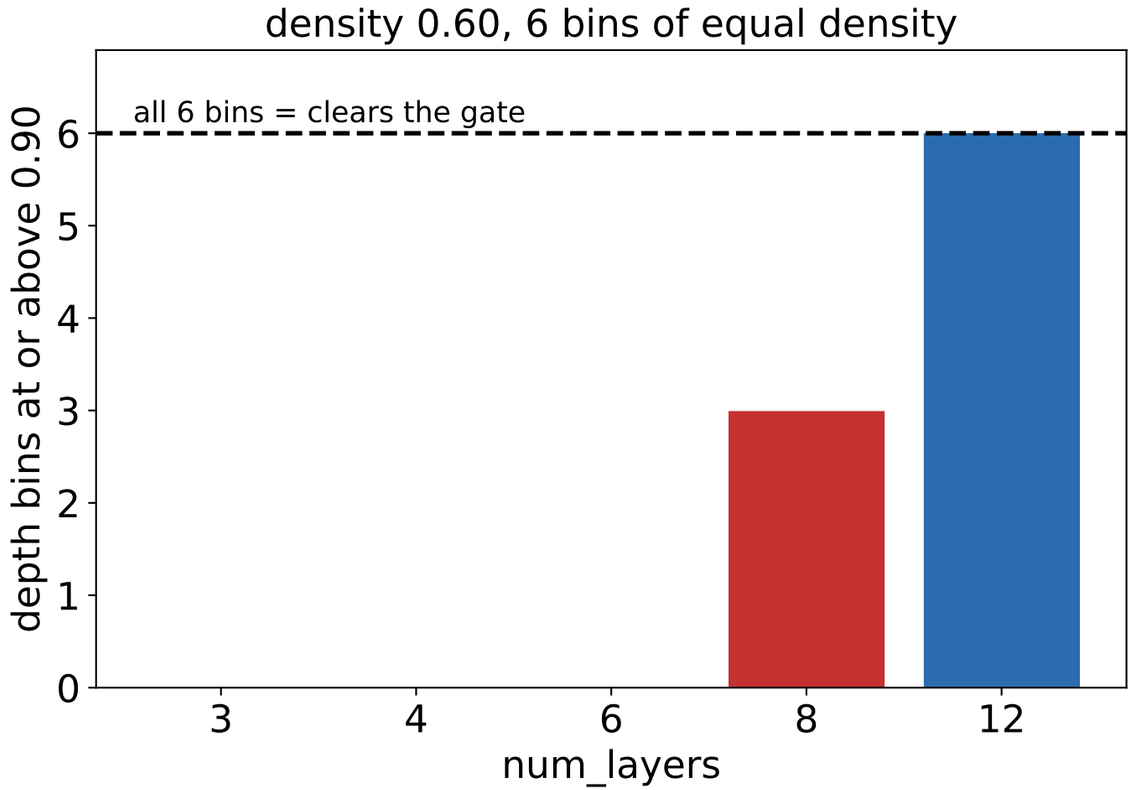
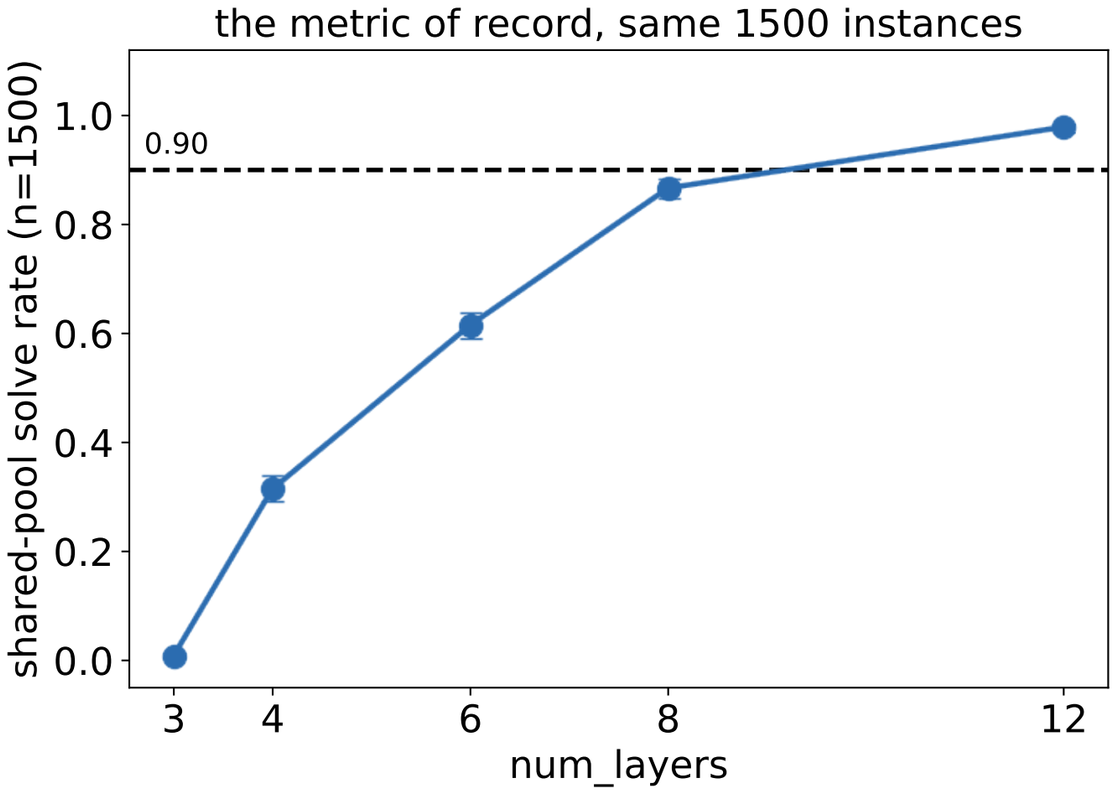
How many layers does the honest N=48 task need?

Leak-free (adjacency shuffled), 240000 instances x 100 epochs, ff=1024.
Two weeks ago the answer was "more than 6, we do not know how many".

L* lies in (8, 12]. The ladder is closed.





Jobs 50527633 (L=12) and 50527634 (L=8), commit 4b9c0e5, seed 0; L=6/4/3 are finding 33's cells (50099945), re-evaluated inside both jobs and identical instance for instance. Shared pool: L=12 0.978, L=8 0.866, L=6 0.615, L=4 0.315, L=3 0.009.

What is being measured, and what changed

One number per cell is not enough: L* is a statement about depth bins, not about an average.

THE TASK IS THE LEAK-FREE ONE.

Finding 16 showed the generator listed each node's neighbours sorted by their true colour, which let a model read the answer off token ORDER -- every leaked-order solve rate in this project is a decoding score. --shuffle-adj removes it. These five cells are all leak-free, so the depth they need is the depth DEDUCTION needs.

L* IS DEFINED ON BINS AT FIXED DENSITY.

Instances are binned by T, the number of DD rounds they actually need, at a requested density of 0.60. A cell clears the gate only if EVERY bin is at or above 0.90. Density is pinned inside the bins -- the realized spread is printed on the bin page -- so "deeper" cannot quietly mean "sparser", which is the confound that invalidated the first version of this measurement.

WHAT CHANGED SINCE 10-29.

Finding 33 ran L = 6, 4, 3 and none of them cleared a single bin, so L* was UNDEFINED above 6 -- the ladder was open at the top and could not be read. These two jobs add L=8 and L=12 going upward, at the same budget and on the same pool, and each job re-scores the 10-29 cells so that every contrast is paired on identical instances.

THE FREE REPRODUCIBILITY CHECK.

Because the reference cells are evaluated two or three times across jobs, the loader asserts the repeats agree instance for instance before pooling anything. They do. A disagreement would have meant the eval path was not deterministic and nothing below would be safe to pair.

Pre-registrations were fixed in the sbatch header before either job landed, and the analysis script was written while they trained, so no threshold here was chosen after seeing a number.

What the model actually sees

Every token is one categorical id into a 58-symbol vocabulary (N=48, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING
	BEGIN_ROUND SOLVED UNRESOLVED
	markers (UNRESOLVED = DD got stuck; unused here)
6-9	COLOR_0 COLOR_1 COLOR_2 COLOR_3
	the q colours (token name COLOR_n, prose 'colour')
10-57	N0 N1 ... N47
	one id per node, no structure in the id

GRAMMAR

BEGIN_HINTS	(NODE COLOR)*
BEGIN_GRAPH	(NODE (NODE){degree})*
BEGIN_REASONING	(BEGIN_ROUND (NODE COLOR)+)*
SOLVED	

*regex: * = zero or more, + = one or more, {degree} = exactly 5*

The graph section is a neighbour table, not an image:
node, then its 5 neighbours.

THE ORDER WITHIN A ROW IS THE LEAK.

THIS INSTANCE, 393 TOKENS

HINTS	BEGIN_HINTS N0 COLOR_1 N2 COLOR_2 N3 COLOR_3 N5 COLOR_0 N8 COLOR_2 N9 COLOR_2 ... (59 tokens)
GRAPH	BEGIN_GRAPH N0 N10 N22 N7 N9 N44 N1 N39 N27 N23 N40 N21 ... (289 tokens)
REASONING	BEGIN_REASONING BEGIN_ROUND N1 COLOR_1 N4 COLOR_3 N7 COLOR_0 N16 COLOR_2 N17 COLOR_3 N18 ... (45 tokens)

TRAINING LOSS is next-token cross-entropy on EVERY token, the problem statement included -- there is no section mask.
AT EVAL the model is PROMPTED with everything through BEGIN_REASONING and generates the rest: 5 rounds, 19 deductions,
38 node+colour tokens, plus the BEGIN_ROUND markers and the SOLVED/UNRESOLVED terminator, which are trained targets like any other token.

DD is single-candidate elimination -- sudoku's 'naked single' rule transplanted to graph colouring, which is where the code's sudoku names come from; the task itself is the graph, not a 9x9 grid. Instance shown: seed 50000000023.

One real reasoning trace

29 hints of 48 nodes, then 5 rounds of simultaneous deduction. Hints: N0=COLOR_1 N2=COLOR_2 N3=COLOR_3 N5=COLOR_0 N8=COLOR_2 N9=COLOR_2 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N1 COLOR_1 N4 COLOR_3 N7 COLOR_0 N16 COLOR_2 ...
  10 deduced
BEGIN_ROUND          N6 COLOR_0 N24 COLOR_0 N27 COLOR_3 N31 COLOR_0 ...
  5 deduced
BEGIN_ROUND          N26 COLOR_2 N41 COLOR_1
  2 deduced
BEGIN_ROUND          N10 COLOR_3
  1 deduced
... 1 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N1, neighbours N39 N27 N23 N40 N21

their colours known before this round:

```
   [0, 2, 3]  (3 = q-1 of the 4)

   (4 known neighbours, 3 DISTINCT colours -- some repeat)

   still unknown: N27
```

only colour left: 1

```
so the trace emits  N1 COLOR_1
```

The model is never shown this reasoning -- only the node/colour pair.

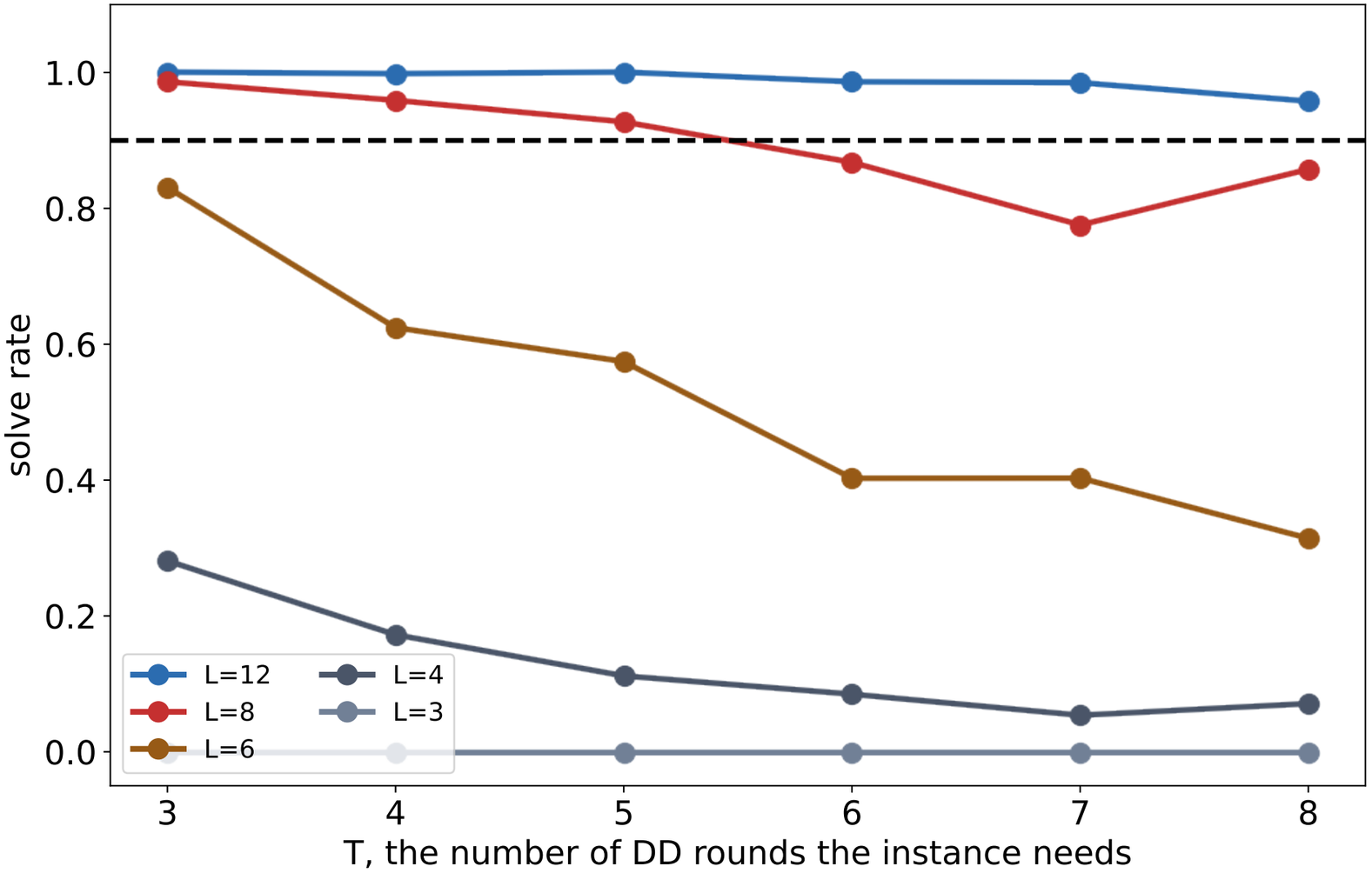
A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

T is the x-axis of every plot in this deck: it is the number of DD ROUNDS this instance needs, counted on the trace above -- not the number of tokens and not the number of nodes. The depth bins group held-out instances by exactly this number, at fixed density, and L* asks how many layers the network needs before it clears 0.90 in all of them.

T=5 rounds, the 90th percentile of 40 draws from this config (range 1-9) -- one instance, shown to make the task legible; no result in this deck is computed from it. The worked deduction on the right uses the state BEFORE this round, which is what DD itself saw; it is recomputed from the graph, not read out of the model.

The gate, bin by bin, at density 0.60

Each line is one cell. Dashed = the 0.90 gate. A cell owns L* only if its whole line is above it.




```
bins cleared, of 6

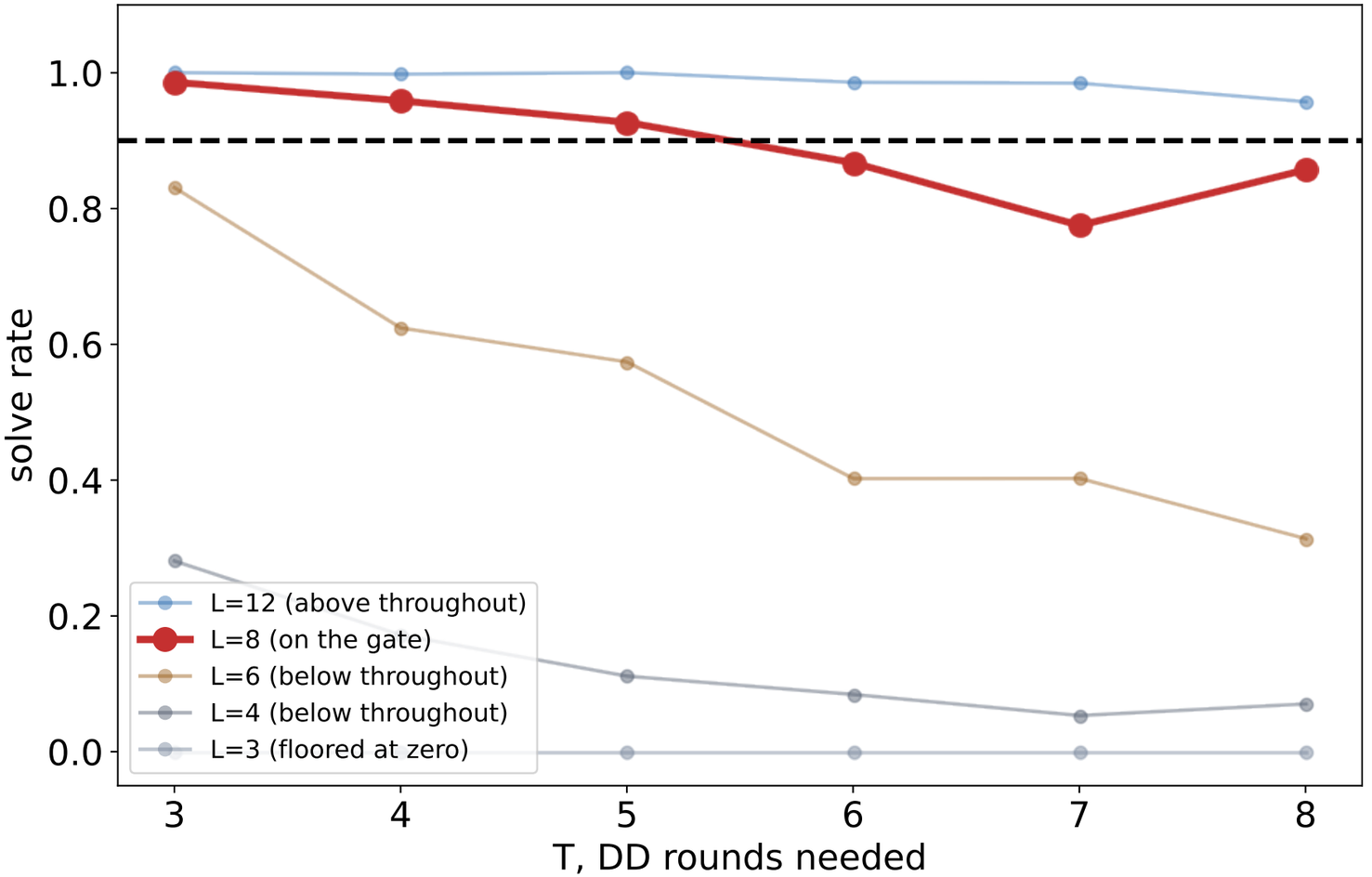
   L=12  6/6   clears
   L=8   3/6   <-- on the gate
   L=6   0/6
   L=4   0/6
   L=3   0/6

realized density spread
   0.000488 across all bins
   (the control: deeper is not sparser)
```

Bins with fewer than ~60 instances are dropped by the eval, so the T range shown is the range the pool actually supports at this density.

Does the depth the network needs track the depth the problem needs?

YES, and it shows up twice. L=8 clears the three shallowest bins and misses the three deepest, so its gate crosses between T=5 and T=6. And L=12, which clears every bin, still falls 1.000 -> 0.957 with z = -4.00: it is above the line, not flat.



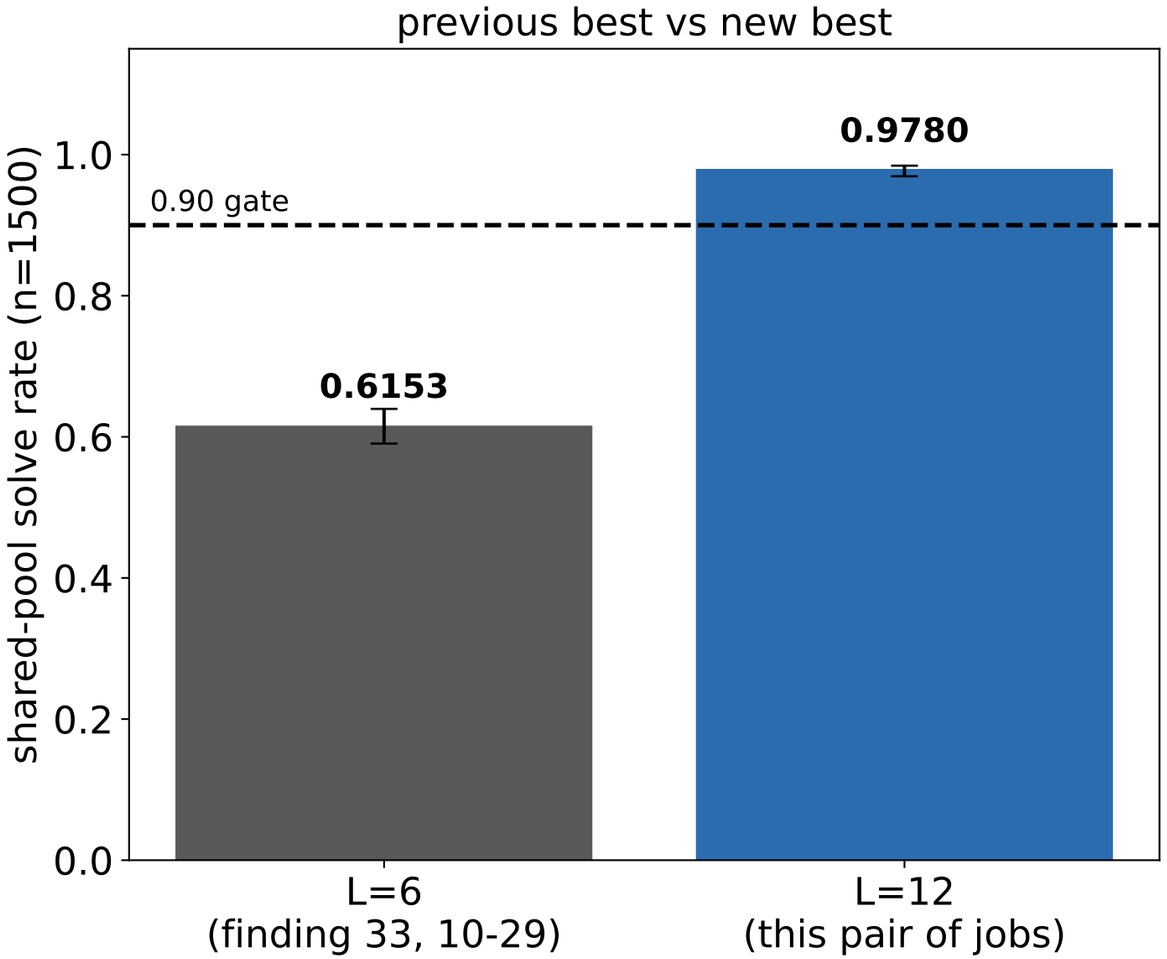
cell	top	bottom	C-A z	where it sits
L=12	1.000	0.957	-4.00	above throughout
L=8	0.986	0.775	-7.65	ON THE GATE
L=6	0.831	0.314	-10.51	below throughout
L=4	0.282	0.054	-6.77	below throughout
L=3	0.000	0.000	+nan	floored at zero

Every cell that is not pinned at zero falls with T. A cell at the floor CANNOT trend, whatever the model does -- which is why findings 11 and 33 could not answer this and finding 22 could only answer it marginally (z = -1.92, p = 0.055).

Cochran-Armitage is a trend test across ordered bins; negative means solve rate falls as T rises. It is reported for every cell, so the one pinned at the floor can be seen not to move. L=8's T=7 bin (0.775, n=129) is below its T=8 bin (0.857, n=70): the decline is a trend, not a monotone staircase.

Against the previous best on the same eval

Same 1500 instances, same protocol, so this is a paired test and not a comparison of two averages.




```
paired delta      +0.3627
discordant        546 - 2
McNemar exact p   3.27e-160

L* at N=48, by task and MLP width

  leaked order, ff=1024    L* = 4
  leaked order, ff=256     L* = 6
  LEAK-FREE,    ff=1024    L* in (8, 12]

Removing the adjacency-order leak costs
at least FOUR rungs of depth at N=48,
and that is measured at 3x the data.

The leaked rows sit on a DIFFERENT pool
(shuffle_adj is in INSTANCE_KEYS), so they
are quoted for context and not tested.
```

Finding 33's L=6 is the previous best because it was the top of the old ladder; L=4 and L=3 score far lower and are shown on the bin page.

Pre-registrations scored, and where this is not robust

Fixed in the sbatch header before either job landed.

(a) L=12 CLEARS EVERY BIN, SO L* IS DEFINED. CONFIRMED.

6 of 6 bins at density 0.60. This is the whole point of the pair of jobs: finding 33 left the ladder open at the top, and it is now closed.

(b) L=8 LANDS IN [0.70, 0.84] ON THE POOL. MISSED, ON THE HIGH SIDE.

It scored 0.8660. The interval came from interpolating finding 33's L=6 toward finding 23's N=24 ceiling, and that interpolation was too pessimistic. My data-and-depth predictions in this project remain mostly underestimates, which is worth saying out loud.

(c) L*(T) RISES AT WHICHEVER CELL SITS ON THE GATE. CONFIRMED AT L=8.

The three bins it misses are the three deepest, so the gate crosses between T=5 and T=6. This is the first time the leak-free task has produced a cell on the gate at all, and the cleanest version of the result finding 22 could only get marginally.

WHERE IT IS NOT ROBUST.

ONE SEED per cell -- finding 37 has just shown a second seed moving a layer-indexed readout by one rung elsewhere in this project. THE BRACKET IS THREE RUNGS WIDE: L=9, 10, 11 are unmeasured, so one L=10 cell halves it and is the cheapest next job. ONLY ONE CELL STRADDLES THE GATE, so the crossing point T=5/T=6 rests on L=8's six bins alone, and its T=7 bin (0.775) sits below its T=8 bin (0.857). FAST PHASE ONLY (finding 34). And everything here is at ff=1024, so the leak-free width factor at N=48 is unmeasured -- that is the hyper-parameter scan this ladder was built for.

Notebook: notebooks/sudoku_n48deep_2026-10-31_results.ipynb. Analysis: $SCRATCH/n48_deep_analysis.py, written while the jobs trained and validated against the 10-29 ladder it reproduces exactly.